Speaker Notes

Potential-Based Reward Shaping for Planar Pushing
Single-Agent Control vs. Asymmetric Self-Play

Vlad Iftime (s3426394)

June 2026 — University of Groningen, FSE

This handout reproduces the slide speaker notes verbatim. Slides that contain mathematics also carry a *Formula breakdown* defining every symbol and stating what the maths means.

Slide 1 — Title Page

- Welcome — 30-minute presentation on my master thesis
- Topic: using potential-based reward shaping to train a UR5e robot arm to push objects to goal poses
- I built 8 model variants and systematically compared them
- Key finding: the simplest model wins — PBRS single-agent achieves 80% validation success
- All attempts to add complexity (curriculum, adversarial self-play) make things worse
- I'll explain why and what this means for RL in contact-rich manipulation

---

Slide 2 — Overview: Thesis Structure

- This is the roadmap for the whole talk — four stages, left to right
- Stage 1, Planar Pushing MDP: I define the task, the quasi-static push mechanics (limit surface), and the SE(2) distance metric
- Stage 2, PBRS Theory: why the old ad-hoc reward failed, then the potential-based reward shaping theory and our potential functions
- Stage 3, 8 Model Variants: Model A (PBRS single-agent, our best), Model B (curriculum), and Models C–H (asymmetric self-play)
- Stage 4, Key Findings: why ASP fails, the central independence result, computational overhead, and the overall comparison and conclusions
- Next slide states the three research questions that this roadmap answers

---

Slide 3 — Research Questions

- Three questions structure the whole presentation
- RQ1: Yes — Model A achieves 80% validation SR, 4.6× the best ad-hoc baseline
- RQ2: No — curriculum (Model B) and ASP (Models C-H) both underperform the simple single-agent
- RQ3: Yes — and this is the central contribution. The clean ablation shows they are independent levers

- Roadmap: I'll define the MDP, explain why PBRS was needed, walk through each model, and present the comparison
- 8 models, 3000 iterations each, 528–2048 parallel environments on an RTX Pro 6000

Slide 4 — MDP Formulation: State & Action

- The MDP is a planar pushing task: a UR5e robot pushes a T-block or disc object on a 1.4m × 1.0m table
- State is 28D: end-effector pose (6), object state including position and quaternion (14), goal pose (6), and current distance errors (2)
- Action is 4D macro-parameterized as object-relative approach (r, phi) plus world-frame push direction (length, theta). 21 bins per dimension = 194k discrete actions
- Object-relative actions are critical — they guarantee 95% contact rate regardless of object spawn position, vs 2% for absolute coordinates

---

Formula breakdown

$$\mathbf{s} = \left[\ \mathbf{q}_{\text{ee}}(6)\ \middle|\ \mathbf{p}_{\text{obj}}, \mathbf{R}_{\text{obj}}(14)\ \middle|\ \mathbf{p}_{\text{goal}}, \mathbf{R}_{\text{goal}}(6)\ \middle|\ d_{\text{pos}}, d_{\text{rot}}(2)\ \right] \in \mathbb{R}^{28}$$

Terms:

- $\mathbf{q}_{\text{ee}}$ — end-effector pose (position + orientation), 6 numbers
- $\mathbf{p}_{\text{obj}}, \mathbf{R}_{\text{obj}}$ — object position and orientation state, 14 numbers
- $\mathbf{p}_{\text{goal}}, \mathbf{R}_{\text{goal}}$ — target pose the object must reach, 6 numbers
- $d_{\text{pos}}, d_{\text{rot}}$ — current position and rotation error to the goal, 2 numbers

Meaning: the complete observation the policy reads each step — everything needed to choose the next push.

$$\mathbf{a} = (r, \phi, \ell, \theta), \qquad 21^4 = 194{,}481 \text{ discrete actions}$$

$$X_s = x_{\text{obj}} + r\cos(\psi_{\text{obj}} + \phi), \quad Y_s = y_{\text{obj}} + r\sin(\psi_{\text{obj}} + \phi), \quad X_f = X_s + \ell\cos\theta, \quad Y_f = Y_s + \ell\sin\theta$$

Terms:

- r — approach radius from the object centre (where to touch); ϕ — approach angle in the object frame
- ℓ — push length; θ — push direction in the world frame
- ψ_{obj} — object yaw; (X_s, Y_s) push start, (X_f, Y_f) push end; 21 bins per dimension

Meaning: one push is a macro-action: pick a contact point on the object (r, ϕ), then shove it a distance ℓ in direction θ. Defining it relative to the object guarantees the gripper actually contacts it.

Slide 5 — MDP Formulation: Transition & Episode

- The physics is quasi-static: the object motion is determined by the limit surface normality theorem — every push couples translation and rotation
- Episode runs max 15 pushes. Terminates on combined success (pos $<$ 5cm AND rot $<$ 0.2 rad) or on catastrophe (tipped, launched, out-of-workspace)
- The table is 1.40m × 1.00m, IK workspace X $\in [-0.50, 0.50]$, Y $\in [0.25, 0.70]$

- GIF suggestion shows a real robot doing 3 pushes to reach a goal — this grounds the abstract MDP in the physical task

---

Formula breakdown

$$\mathbf{t} = (v_x, v_y, \omega) \propto \nabla f(\mathbf{w})$$

Terms:

- $\mathbf{t}$ — the object twist: planar linear velocity (v_x, v_y) plus angular velocity ω
- $f(\mathbf{w})$ — the limit surface, the boundary in wrench space $\mathbf{w} = (\text{force}, \text{torque})$ the contact can sustain
- ∇f — gradient (the surface normal direction)

Meaning: the resulting motion points along the surface normal, so a single push almost always produces translation *and* rotation at once — you cannot command one without the other.

Slide 6 — Push Mechanics: Why Position & Rotation Are Coupled

- The central physics insight: in quasi-static planar pushing, translation and rotation are mechanically coupled through the limit surface
- The limit surface is a convex hull in wrench space — all force-torque combinations the contact patch can support
- Because the surface is convex and smooth, the motion direction (twist) is always normal to the surface at the applied wrench point
- This means no ”push forward without rotating” or ”rotate without translating” — every push is a coupled SE(2) motion
- The characteristic length L is the radius of gyration of the pressure distribution. T-block L=0.07m (strong coupling), disc L=0m (decoupled)

Formula breakdown

$$\mathbf{t} \propto \nabla f(\mathbf{w}), \qquad L = \text{radius of gyration}$$

Terms:

- $\mathbf{t} \propto \nabla f(\mathbf{w})$ — normality law (same as previous slide): twist is normal to the limit surface
- L — characteristic length / radius of gyration of the object's pressure distribution (metres)
- T-block $L = 0.07\,\text{m}$ (rotation strongly couples to translation); disc $L = 0\,\text{m}$ (rotation decoupled / irrelevant)

Meaning: L quantifies *how much* a push rotates the object; it is fixed by the object's mass/contact geometry, not chosen — which is why the reward can use it as a physical constant rather than a tuned knob.

Slide 7 — SE(2) Unified Distance (Models E–H)

- The SE(2) unified distance replaces separate position and rotation errors with a single Riemannian metric on the planar rigid-body configuration space
- L converts radians to meters. For the T-block with L=0.07m, 0.5 rad of rotation contributes 3.5cm to d_pose — roughly equal weight with a 3.5cm position displacement

- For the disc with L=0m, rotation is irrelevant — d_pose collapses to pure position distance, matching the physical symmetry
- The single-potential PBRS eliminates the competing gradient problem: separate pos/rot potentials each pull in different directions, while the SE(2) potential provides a unified gradient
- Both Model E (SE(2) d_pose ASP) and Model C (separate ASP) collapse — the SE(2) metric doesn't rescue the self-play dynamics, confirming the curriculum is the root cause, not the reward metric

---

Formula breakdown

$$d_{\text{pose}} = \sqrt{dx^2 + dy^2 + L^2\, d\theta^2} \qquad \Phi(s) = w\, e^{-k\, d_{\text{pose}}^2}, \quad k = 30,\ w = 10$$

Terms:

- dx, dy — object→goal position error in x, y (metres); $d\theta$ — yaw error (radians)
- L — characteristic length (metres), converts the radian error into metres so it can be added to dx, dy
- d_{pose} — single SE(2) distance to the goal (metres)
- $\Phi(s)$ — potential (state "goodness"); w — peak height; k — decay sharpness

Meaning: d_{pose} is one scalar that fuses position and orientation error, with L as the exchange rate between rotating and translating; Φ turns that distance into a smooth bump peaking at the goal, so position and rotation are optimised by one gradient instead of two competing ones.

Slide 8 — Old Reward Formula: Why It Failed

- The old formula had three fatal flaws that collectively explain why the Push-PPO baseline plateaued at 17%
- First, it's non-Markov: dividing by d_prev means reward depends not just on s and s' but on the full history. The same state transition gets different rewards from different starting positions. This violates the Markov property that underlies all RL theory. In my thesis appendix, I prove that no static potential function Phi(s) can reproduce this reward
- Second, gradient starvation: a push that genuinely produces progress (1cm closer from 30cm away) gets penalized because the penalty term beta·d_now dominates the tiny improvement term. The expected reward for random pushes is negative with near-zero variance — PPO's GAE advantage estimator gets literally no signal to differentiate good from bad pushes
- Third, in earlier versions (before P63), independent alpha_pos=12 and alpha_rot=1 caused rotation gradients to dominate position 20×, resulting in RotSR=42% while PosErr stayed flat at 0.25m — the agent learned to rotate but never learned to translate
- The end result: 17.3% training SR, 0.78 rad rot error (essentially random rotation). This is WHY we needed a better reward function

---

Formula breakdown

$$R = \alpha\, \frac{d_{\text{prev}} - d_{\text{now}}}{d_{\text{prev}}} + \alpha\, \frac{y_{\text{prev}} - y_{\text{now}}}{y_{\text{prev}}} - \beta\, d_{\text{now}} - \beta_{\text{rot}}\, y_{\text{now}}, \qquad \alpha = 3.0,\ \beta = 0.5,\ \beta_{\text{rot}} = 0.25$$

Terms:

- d — position error, y — rotation error; subscripts prev/now = before / after the push

- α — gain on fractional improvement; $\beta, \beta_{\text{rot}}$ — penalties on the error still remaining

Meaning: it pays for *fractional* progress minus a standing penalty — but dividing by d_{prev} makes the same transition worth different amounts depending on where you started (non-Markov), and the penalty term swamps small honest gains.

$$\underbrace{3 \cdot \tfrac{0.01}{0.29}}_{\text{improvement}} - \underbrace{0.5 \cdot 0.29}_{\text{penalty}} = -0.025, \qquad \mathbb{E}[R] \approx -0.15$$

Meaning: a genuinely good 1 cm push from 30 cm away scores *negative*; random pushes average ≈ -0.15 with almost no variance, so PPO's advantage estimator sees no signal to tell good pushes from bad — the agent never learns.

Slide 9 — PBRS: The Shaping Theorem

- The fundamental theorem: add any potential-based shaping F = gamma·Phi(s') - Phi(s) without changing the set of optimal policies
- Proof: Q'(s,a) = Q(s,a) - Phi(s). Since Phi(s) depends only on state (not action), subtracting it from Q doesn't change which action maximizes Q
- Episodic PBRS (Grześ 2017): gamma_shaping=1.0 is valid because the sum over an episode is bounded
- This decoupling of reward from policy is the core insight — you can design rewards purely for learning speed without worrying about distorting the objective

---

Formula breakdown

$$\mathcal{R}'(s,a,s') = \mathcal{R}(s,a,s') + \gamma\,\Phi(s') - \Phi(s) \qquad \Longrightarrow \qquad Q'(s,a) = Q(s,a) - \Phi(s)$$

Terms:

- $\mathcal{R}$ — original (true) reward; $\mathcal{R}'$ — shaped reward the agent actually trains on
- γ — discount factor; $\Phi(s)$ — any function of the state alone (the potential)
- Q, Q' — action-values under the original / shaped reward

Meaning: adding exactly $\gamma\Phi(s') - \Phi(s)$ shifts every action's value by $-\Phi(s)$, which does not depend on the action a; so the best action (the $\arg\max$) is unchanged. The reward can be redesigned to speed up learning while provably keeping the same optimal policy.

$$\sum_t \big(\gamma\Phi(s_{t+1}) - \Phi(s_t)\big) = \Phi(s_T) - \Phi(s_0) \quad \text{(episodic, } \gamma_{\text{shaping}} = 1\text{)}$$

Meaning: over a complete episode the shaping telescopes to a fixed constant, so it is bounded and cannot be "farmed" — which is why $\gamma_{\text{shaping}} = 1$ is safe for our finite-horizon task.

Slide 10 — Why PBRS Works for Planar Pushing

- Six properties: policy invariance means you don't have to worry about the shaping distorting the optimal behavior
- Markovian: the reward depends only on s and s', not on history. Old formula violated this by dividing by d_prev
- Zero-sum cycles: detour pushes produce zero net F — the agent isn't penalized for exploratory movements

- No constant drain: unlike the old formula which had a beta·d_now penalty, if the agent doesn't move, F=0

Formula breakdown

$$F(s, s') = \gamma\,\Phi(s') - \Phi(s)$$

Terms:

- F — the dense shaping reward added on every push; $\Phi(s'), \Phi(s)$ — potential after / before the push

Meaning: the reward depends only on the change in potential between two states — hence Markovian, exactly zero around any loop (no reward hacking), and exactly zero when nothing moves (no penalty for thinking).

Slide 11 — Why PBRS Works: The Reward Diet

- The reward diet plot shows the agent gets 85% of its reward from dense PBRS shaping, with sparse bonuses and penalties as supplementary signals
- Because the dense signal fires every push and is bounded, the critic stays stable and PPO always has a gradient to follow

Slide 12 — PBRS: Our Potential Functions

- We use two potential variants: separate position/rotation (Models A-C) and unified SE(2) (Models E-H)
- The separate variant uses exponential potentials: k_p=30 makes the potential decay sharply — near the goal the gradient is strong, far away it's weak
- Cosine angular distance c(s) ∈ [0,1] is chosen over raw yaw error because it's smooth and bounded — no discontinuity at ±pi
- Dense reward is purely difference-based: F = Phi(s') - Phi(s). Sparse completion bonuses (+5 pos, +2 both-threshold) fire on the final push

Formula breakdown

$$\Phi(s) = w_{\text{pos}}\, e^{-k_p\, d_{\text{pos}}^2} + w_{\text{rot}}\, e^{-k_r\, c(s)}, \qquad c(s) = \frac{1 - \cos(\theta^* - \theta)}{2} \in [0, 1]$$

$$k_p = 30,\ k_r = 5,\ w_{\text{pos}} = w_{\text{rot}} = 10.0$$

Terms:

- two bumps: one for position (d_{pos}), one for rotation ($c(s)$)
- $c(s)$ — cosine angular distance; θ^* — goal yaw, θ — current yaw
- k_p, k_r — decay sharpness of each bump; $w_{\text{pos}}, w_{\text{rot}}$ — their weights

Meaning: two independent smooth potentials reward getting closer in position and in orientation; the cosine form keeps the rotation term continuous across the $\pm\pi$ wrap-around, so there is no artificial cliff at 180°.

Slide 13 — PBRS: Why Our Potentials Work

- Bounded exponential potentials prevent reward explosions from PhysX glitches; the potential difference gives a natural gradient toward the goal
- Cosine distance respects the S^1 topology of rotation
- These hyperparameters are robust: 3 independent training runs with identical settings all converged to similar performance

Formula breakdown

$$F = \Phi(s') - \Phi(s), \qquad 0 \le e^{-k\,d^2} \le 1$$

Meaning: because each exponential is bounded in $[0, 1]$, the per-step reward is bounded too — physics spikes cannot blow up the return — while $\Phi(s') - \Phi(s)$ still points uphill toward the goal. The cosine term treats yaw as living on a circle (S^1), so $-\pi$ and $+\pi$ are the same point.

Slide 14 — Model A: PBRS Single-Agent (Our Best Model)

- Model A is the simplest possible architecture: single PPO agent with LSTM and MLP, PBRS reward, sparse completion bonuses
- Three reasons it works: (1) PBRS provides dense gradient at every push — unlike ASP where sparse bonuses fire on 0.14% of pushes, (2) fixed random goal distribution — no adversarial shift breaking PPO, (3) single network — no multi-agent optimization instability
- Converges within 1500 iterations. Stable value function.

Formula breakdown

$$\text{LSTM}(28 \to 256)\text{+MLP}[512, 256], \quad \text{head } 4\times 21; \qquad \gamma = 0.998,\ \lambda = 0.95,\ \varepsilon = 0.2,\ \text{ent\_coef} = 0.005$$

Terms:

- $28 \to 256$ — 28-D observation encoded into a 256-D recurrent (LSTM) state; MLP widths 512, 256
- head 4×21 — the 4 action dimensions, each a 21-way categorical
- γ — discount (near 1 = long horizon); λ — GAE smoothing; ε — PPO clip width; ent_coef — entropy-bonus weight

Meaning: a standard recurrent PPO agent; $\gamma = 0.998$ values the whole episode, $\varepsilon = 0.2$ sets the trust-region size, and a small entropy coefficient gives mild exploration without drowning the gradient.

Slide 15 — Model A: Training Dynamics

- Left plot: training success rate over 3000 iterations. Model A (blue) steadily improves from near-zero to 8.75% (strict combined pos+rot gate). Models B-H plateau at near-zero or diverge
- Right plot: value loss stays stable at 8-12 — no explosion. Earlier attempts with gain=1.0 on the critic caused Val loss 27k-356k (Fix P29). Gain=0.01 keeps the critic stable
- The PBRS exponential potentials prevent the reward spikes that caused GAE chain reactions and critic instability

Formula breakdown

$$\text{critic output gain} = 0.01 \;\Rightarrow\; V_{\text{init}} \approx 0.057$$

Terms:

- gain — the initial scale of the critic's final linear layer; V_{init} — its value prediction at iteration 0

Meaning: a tiny initial gain keeps the critic's first guesses near zero, so the GAE return estimates start small and bounded instead of exploding into the hundreds of thousands (the failure seen with gain = 1.0).

Slide 16 — Model A: Validation Results

- Validation: 30 held-out scenes — 10 disc, 10 T-block pos-only, 10 T-block pos+rot. 80% overall SR
- Perfect on disc objects (100%), very strong on T-block pos-only (80%), respectable on hardest pos+rot scenes (60%)
- Quantitative comparison vs Push-PPO baseline: 3.3× lower PosErr, 3.7× lower RotErr, 4.6× higher validation SR
- The training SR (8.75%) understates validation because the strict combined gate penalizes near-misses in training. Validation uses best-of-3
- The overhead clip is the trained Model A policy recorded top-down (record_push_video.py)

Slide 17 — Model A: Validation Runs (overhead)

- Three representative validation runs recorded top-down with record_push_video.py
- Left: a short forward push (E_Forward, position-only)
- Middle: a lateral push (E_Left, position-only)
- Right: a combined translate-and-rotate scene (E_Diag, position+rotation)
- All use the same simple PBRS Model A checkpoint with object-relative obs+actions

---

Slide 18 — Model A: Validation Breakdown

(no speaker notes yet)

Slide 19 — Model B: PBRS + Forced Curriculum (Broken)

- Model B was designed to learn position first, then progressively add rotation — a classic curriculum approach
- But the curriculum never advanced past phase 1. w_rot stayed at 0.0 the whole run — the agent trained position-only the entire time
- Root cause: PBRS already provides dense, informative gradients. The agent continuously improves without plateauing, so the EMA never stabilizes low enough to trigger the phase transition

- When the reward is already good, a curriculum controller adds overhead without benefit — and here it actively prevented rotation learning

---

Formula breakdown

$$\text{trigger Phase 2 when} \quad \text{EMA}_{\text{pos\_err}} < 0.08\,\text{m for 50 iters}, \qquad w_{\text{rot}} : 0 \to 10 \text{ over 200 iters}$$

Terms:

- $\text{EMA}_{\text{pos\_err}}$ — exponential moving average of position error (the plateau detector)
- w_{rot} — weight on the rotation potential; 0 = position-only, 10 = full rotation objective

Meaning: the curriculum was meant to switch on rotation once position stabilised below 8 cm; but because PBRS keeps the agent improving, the EMA never settles, the trigger never fires, w_{rot} stays 0, and rotation is never learned.

Slide 20 — Model B: Results (Broken)

- Validation pos-only SR is 70% (reasonable) but pos+rot SR is only 10% — the agent literally never learned rotation
- RotError is 0.503 rad — 2.4× worse than Model A
- This is the first piece of evidence that reward design and curriculum design are independent problems. A good reward makes curriculum unnecessary; a bad curriculum can override a good reward

---

Slide 21 — Models C–D: ASP Architecture (Alice/Bob)

- The ASP architecture: Alice (goal proposer) and Bob (goal achiever) play an adversarial game. Alice gets +5 when Bob fails, -1 when Bob succeeds
- Bob sees Alice's final object pose as his goal. He gets PBRS dense reward + sparse completion bonuses
- When Bob fails, Alice's trajectory is stored in an ABC buffer for behavioral cloning with beta=0.5
- Model D ablates the GoalEncoder to test whether the 8D bottleneck helps or hurts

---

Formula breakdown

$$R_A = \begin{cases} +5 & \text{Bob fails} \\ -1 & \text{Bob succeeds} \end{cases} \qquad R_B = \underbrace{F = \Phi(s') - \Phi(s)}_{\text{dense PBRS}} + \underbrace{(+5/+2)}_{\text{sparse}}, \qquad \beta_{\text{ABC}} = 0.5$$

Terms:

- R_A — Alice's (proposer's) reward; R_B — Bob's (achiever's) reward
- β_{ABC} — weight of the Alice-Behavioral-Cloning loss that imitates Alice's trajectory on Bob's failures

Meaning: Alice is paid to pose goals Bob cannot reach and Bob is paid to reach them — an adversarial game intended to auto-generate a curriculum; β controls how strongly Bob copies Alice when he fails.

Slide 22 — ASP Diagnostic: Alice vs. Bob

- Plot shows Alice's goal validity rate rising to 79% while Bob's success rate stays at 0.07% — clear evidence the adversarial curriculum prevents combined objective achievement
- Alice's mean displacement converges to 0.17m — she's creating precise, low-displacement goals, not impossible ones
- Individual Bob metrics (66% pos SR, 65% rot SR) confirm Bob HAS the skills — he just can't combine them under ASP's non-stationary goal distribution

---

Slide 23 — Models E–H: ASP Variants

- Six ASP variants tested: original outcome-based (C), GoalEncoder ablated (D), SE(2) d_pose (E/F), time-based Alice (G/H)
- E/F replace the separate position/rotation potentials with a single SE(2) d_pose potential — physics-aligned, eliminates competing gradients
- G/H use Sukhbaatar's time-based Alice reward, a self-regulating curriculum that rewards Alice for goals Bob takes longer to solve. ABC is disabled
- The hypothesis was that either change might rescue ASP — neither does

Formula breakdown

$$\text{E/F:}\quad \Phi(s) = w\, e^{-k\, d_{\text{pose}}^2} \qquad\qquad \text{G/H:}\quad R_A = \gamma_{sp}\, \max(0,\ t_B - t_A),\quad \gamma_{sp} = 0.5$$

Terms:

- Φ with d_{pose} — the single SE(2) potential from Slide 7 (replaces the two competing potentials)
- R_A — Alice's time-based reward; t_A — pushes Alice used; t_B — pushes Bob used (or the cap on failure); γ_{sp} — scale

Meaning: two attempted rescues: (E/F) make the reward physics-aligned so position and rotation share one gradient; (G/H) pay Alice for the *effort gap* $t_B - t_A$, so she is rewarded for goals that are hard for Bob but cheap for her — a self-regulating curriculum.

Slide 24 — Models E–H: Results (All Collapse)

- All collapse to 0-7% validation SR — 11-27× below Model A's 80%
- Individual metrics are informative: Bob CAN push to the goal position (66% SR) OR the goal orientation (65% SR) individually — but the combined gate almost never fires
- Time-based ASP (Models G/H) shows marginal improvement (6.7%) — Sukhbaatar's self-regulation partially corrects the adversarial curriculum by rewarding Alice for solvable goals
- But even time-based ASP can't rescue the three structural failure modes: non-stationary distribution + sparse reward starvation + vanishing ABC gradients

Formula breakdown

$$R_A \propto \max(0,\ t_B - t_A)$$

Meaning: Alice earns more the longer Bob struggles (t_B large) relative to her own effort (t_A small); in principle this self-limits goal difficulty, but in practice it only nudges results from ∼0% to ∼7% — the structural failures dominate.

Slide 25 — Why ASP Fails: Failure Modes 1 & 2

- The three failure modes form a vicious cycle. The first two are on this slide
- First, non-stationary goal distribution: Alice learns continuously, so Bob's training distribution shifts every iteration. PPO computes the importance ratio using old_logp from a rollout collected under a DIFFERENT Alice. The ratio is dominated by Alice's distribution shift, not Bob's weight updates. PPO's clip fires on 80% of transitions, killing the gradient
- Second, sparse reward starvation: Bob's sparse bonuses (+1/-1/+5) fire on 0.14% of pushes. GAE advantages = zero-mean noise. The agent cannot distinguish good pushes from bad pushes

Formula breakdown

$$\text{ratio} = \frac{\pi_{\text{new}}(a \mid s)}{\pi_{\text{old}}(a \mid s)} \quad \text{clipped to } [1-\varepsilon,\, 1+\varepsilon], \qquad \varepsilon = 0.2$$

Terms:

- $\pi_{\text{new}}, \pi_{\text{old}}$ — current and rollout-time policies; ratio measures how much the policy changed
- ε — PPO clip width: updates outside $[0.8, 1.2]$ are clipped (gradient zeroed)

Meaning: when Alice keeps changing the goals, the ratio reflects the shifting goal distribution rather than Bob's own update, so it lands outside the clip band ∼80% of the time and the gradient is thrown away. Add to that sparse bonuses firing on 0.14% of pushes, and the advantage signal is just zero-mean noise.

Slide 26 — Why ASP Fails: Failure Mode 3 & Combined Effect

- Third, vanishing ABC gradient: ABC should provide a supervised signal when Bob fails — clone Alice's trajectory. But Alice's actions are high-entropy random walks because her entropy bonus (0.005 × 14 nats) dominates her surrogate loss (0.005). The bc_ratio ≈ 1.0 for all trajectories, providing zero gradient direction
- The net effect: Bob receives no valid gradient from any of the three intended sources. His policy stays at random initialization forever. Evidence: Bob surrogate loss ≈ 0.001 for all 3000 iterations — the model isn't even trying to learn

Formula breakdown

$$\text{bc\_ratio} = \exp\big(\log \pi_{\text{new}} - \log \pi_{\text{old}}\big) \approx 1.0$$

Terms:

- bc_ratio — the imitation (behavioral-cloning) likelihood ratio between Bob's current policy and Alice's recorded actions

Meaning: if Alice acts almost randomly, every trajectory looks equally (im)probable to Bob, so the ratio is ≈ 1 and the cloning loss has no preferred direction — ABC supplies zero learning signal. Combined with modes 1 & 2, Bob gets no gradient from PPO, the environment, or ABC, and never leaves random initialisation.

Slide 27 — Key Result: Reward & Curriculum Are Independent

- This is the central contribution of the thesis — and it's captured cleanly in this 2×2 matrix
- The field often treats reward shaping and curriculum learning as interchangeable tools for the same problem: sparse gradients and exploration. Our clean ablation shows they operate on fundamentally different axes

- Switching from ad-hoc reward to PBRS improves performance dramatically: Push-PPO 17.3% → Model A 80% — a 4.6× improvement from reward alone
- But switching from single-agent to ASP collapses performance regardless of which reward is used: Model A 80% → Model C 0%, Push-PPO 17.3% → Push-ASP 0.07%
- Since PBRS is held constant across Model A and Model C, the failure is cleanly isolated to the self-play curriculum — not the reward
- Practical implication: when your RL system fails, you need to diagnose whether the reward doesn't guide the agent OR whether the training distribution doesn't support learning. Our ablation framework provides the methodology for this diagnosis
- The independence also means you cannot compensate for a bad curriculum with a good reward, nor for a bad reward with a good curriculum — they are independent levers that must be optimized separately

---

Formula breakdown

reward effect: $\frac{80\%}{17.3\%} \approx 4.6\times$ (moves the column); curriculum effect: $80\% \to 0\%$, $17.3\% \to 0.07\%$ (moves the

Meaning: in the 2×2 (reward × curriculum) table, changing the reward shifts results along one axis and changing the curriculum shifts them along the other; the two effects don't interact, so reward design and curriculum design are independent levers that must be tuned separately.

Slide 28 — Computational Overhead: ASP vs Direct Architecture

- Beyond the learning dynamics failure, ASP imposes a significant computational overhead that makes it fundamentally harder to scale
- ASP requires per-iteration: Alice rollout (100 steps), Bob rollout (100–200 steps), two separate PPO updates (each with LSTM hidden-state propagation), ABC buffer population and sampling (GPU sliding window), historical policy pool management (5 snapshots, sampled), and phase synchronization and env staggering — all before the next Alice phase begins
- Compare this to the Table Tennis project's DirectRLEnv approach: a single PPO agent, no managers, no phase management, pure tensor operations. The DirectRLEnv API eliminates per-step Python dispatch through ManagerBasedRLEnv's ActionManager, ObservationManager, EventManager, and TerminationManager
- The result: 20 iterations/second at 4096 envs on an RTX Pro 6000 → 2.1 million transitions per iteration. ASP at 528 envs achieves only a fraction of this throughput
- The architectural cost is independent of the learning dynamics issues — even if ASP eventually converged, you're paying for two agents but getting zero performance benefit
- The Table Tennis project explicitly chose DirectRLEnv because it provides 5–10× faster training throughput. The same design philosophy applies to planar pushing

---

Slide 29 — Comparison Summary: All Models

- This is the unified comparison table — all 10 models across all metrics
- Model A dominates: 80% validation, lowest errors, simplest architecture. No other model comes close
- The ASP variants (C through H) uniformly underperform — no variant exceeds 7%. This is true regardless of reward type, architecture variant, or object type

- The ad-hoc reward models (Push-PPO, Push-ASP) show the same pattern: single-agent (17.3%) beats ASP (0.07%), but both are worse than PBRS equivalents
- Model B is broken by its curriculum trigger — it never advanced past position-only training

---

Slide 30 — Comparison Summary: Failure Taxonomy

- The comparison plot shows training curves over 3000 iterations. Model A (blue) steadily improves. Model B (orange) plateaus early. Models C/D (green/red) barely move
- The overarching pattern: every structural addition — curriculum controller, second agent, GoalEncoder, ABC buffer, historical pool, SE(2) metric — adds failure modes without adding performance. The simplest model wins

---

Slide 31 — What This Disproves

(no speaker notes yet)

Slide 32 — What This Supports & Disproves

- This slide summarizes the theoretical contributions — what the thesis proves and disproves
- Claim 1 (Plappert): ASP generates an intrinsic curriculum for exploration. Our data: ASP generates a toxic curriculum — Alice finds goals where Bob's individual metrics are high (66% pos, 65% rot) but the combined gate is impossible (0.07%). The curriculum is adversarial, not constructive
- Claim 2 (Sukhbaatar): the GoalEncoder bottleneck helps Bob compactly represent the goal. Our data: ablating the GoalEncoder changes nothing. Model D without it performs identically to Model C with it. The 8D compression doesn't help or hurt — it's irrelevant under adversarial curriculum
- Claim 3 (Narvekar/Portelas): curriculum learning helps when the base reward is informative. Our data: Model B's curriculum prevented rotation learning entirely. The EMA-based activation threshold created a degenerate attractor — PBRS's continuous improvement prevented phase transition
- The PBRS theory (Ng, Grzes) is strongly supported. The policy invariance theorem produced robust results — k_p=30, k_r=5, w=10 are not fragile hyperparameters; they work well across 3 independent single-agent training runs

---

Formula breakdown

$$k_p = 30, \quad k_r = 5, \quad w = 10.0$$

Terms:

- k_p, k_r — position / rotation potential decay sharpness; w — potential weight (height)

Meaning: the same three potential constants worked across all single-agent runs and three independent seeds — evidence that PBRS is robust, not a fragile hand-tuned reward.

Slide 33 — Limitations & Research Gaps

(no speaker notes yet)

Slide 34 — Future Work: Promising Directions

- Our results are conclusive within the experimental budget, but they don't close the book on ASP. Several promising directions remain
- Model I/J: The Bob time penalty creates a near-zero-sum game structure: Alice gains proportionally to (t_B - t_A), Bob loses proportionally to t_B. From Letcher et al. (2019), the game Jacobian decomposes into symmetric (potential) and antisymmetric (Hamiltonian) components. Zero-sum games have pure antisymmetric dynamics, which are conservative — they cycle rather than diverge. This could stabilize ASP training where general-sum ASP diverges
- Model K/L: ABC with beta=0.5 after Alice converges. Currently, ABC fails because Alice's actions are random, so the bc_ratio ≈ 1.0 for all trajectories. If Alice converges first (e.g., with time-based reward and disabled Bob), then Bob starts with a meaningful imitation target
- Relaxed success gate: Bob's individual SR is 65-66%, but the combined gate almost never fires. Partial credit for achieving one of the two objectives could provide gradient to eventually learn the combined task
- PAIRED as an alternative: instead of Alice maximizing Bob's failure, the antagonist maximizes the protagonist's regret (difference between protagonist and antagonist returns). This produces solvable but challenging environments — addressing the exact failure mode we observed

---

Formula breakdown

$$R_B \mathrel{+}= -\gamma_{sp}\, t_B$$

Terms:

- R_B — Bob's reward; t_B — number of pushes Bob used; γ_{sp} — the same time-scale used for Alice

Meaning: penalising Bob for taking long makes Alice's gain ($+\gamma_{sp}(t_B - t_A)$) and Bob's loss ($-\gamma_{sp} t_B$) nearly cancel — a near zero-sum game. Game theory (Letcher 2019) says zero-sum dynamics cycle rather than diverge, so this might stabilise the training that general-sum ASP destroys.

Slide 35 — Conclusion (1/2)

(no speaker notes yet)

Slide 36 — Conclusion (2/2)

- Three take-home messages
- First, PBRS works. Model A achieves 80% on the hardest validation suite with the simplest architecture. This is a strong baseline that any future approach should be compared against. The theoretical justification (policy invariance, episodic validity, bounded smooth potentials) is fully supported by the empirical results

- Second, ASP fails structurally. Six independent variants, three reward structures, two architecture variants — all collapse. The individual metrics (66% pos SR, 65% rot SR) show the agents are learning something — but the adversarial curriculum prevents them from ever achieving the combined objective. The failure is not a training budget issue (0.07% after 3000 iters suggests dead gradient, not slow convergence)
- Third, and most importantly, reward and curriculum are independent. This is the methodological contribution. Future work should: (a) design good rewards (PBRS works), (b) diagnose whether the training distribution supports learning, and (c) not assume that adding structure helps
- Practical recommendation: the simplest working system is the best starting point. Our 528-env PBRS single-agent trains in hours, not days, and achieves 80% SR. Start there, and only add complexity when you can prove it helps

---

Appendix

Slide 37 — Appendix Outline

(no speaker notes yet)

Slide 38 — Appendix A: Training Curves (All PBRS Models)

(no speaker notes yet)

Slide 39 — Appendix B: PBRS Theory Analysis

(no speaker notes yet)

Formula breakdown

$$\Phi(s) = w\ \exp(-k\, d^2) \qquad\qquad c(s) = \frac{1 - \cos(\theta^* - \theta)}{2}$$

Terms:

- $\Phi(s)$ — the potential landscape; w — height, k — decay sharpness, d — distance to goal
- $c(s)$ — cosine angular distance; θ^* — goal yaw, θ — current yaw

Meaning: the appendix plots show Φ as a smooth bump peaking at the goal and compares its gradient with the old fractional formula; the cosine curve shows the rotation term is smooth and bounded across the full $[-\pi, \pi]$ range.